Теоретические домашние задания

Теория типов, ИТМО, М3232-М3239, весна 2026 года

Домашнее задание №1: лямбда исчисление — бестиповое и простотипизированное

Для проверки и демонстрации заданий используйте какой-нибудь эмулятор лямбда-исчисления, например LCI: `https://www.chatzi.org/lci/`

1. Постройте лямбда выражения:
   (a) деление чёрчевского нумерала на 3.
   (b) ограниченное вычитание 1 (указание: воспользуйтесь упорядоченными парами).
   (c) целочисленное деление.

2. На лекции был приведён комбинатор неподвижной точки $Y := \lambda f.(\lambda x.f\ (x\ x))\ (\lambda x.f\ (x\ x))$, обладающий свойством $Y\ P =_\beta P\ (Y\ P)$ для любого терма P. С его помощью оказывается возможным реализовывать рекурсию.

   Например, зададим функцию, возводящую 2 в соответствующую степень:

   $$P := \lambda f.\lambda x.(IsZero\ x)\ 1\ ((f\ (Dec\ x)) \cdot 2)$$

   Сравните это с кодом на Си:

   ```
   unsigned f (unsigned x) { return x == 0 ? 1 : f (x-1) * 2; }
   ```

   Тогда, вызванная как $Y\ P\ x$, эта функция вычислит 2^x. Например, $Y\ P\ 1 =_\beta$

   $$\begin{aligned}
   &=_\beta P\ (Y\ P)\ 1 = (\lambda f.\lambda x.(IsZero\ x)\ 1\ ((f(Dec\ x)) \cdot 2)))\ (Y\ P)\ 1\\
   &=_\beta (IsZero\ 1)\ 1\ ((Y\ P\ (Dec\ 1)) \cdot 2))) =_\beta (Y\ P\ 0) \cdot 2\\
   &=_\beta (P\ (Y\ P)\ 0) \cdot 2\\
   &=_\beta (IsZero\ 0)\ 1\ ((Y\ P\ (Dec\ 0)) \cdot 2))) \cdot 2\\
   &=_\beta 1 \cdot 2 =_\beta 2
   \end{aligned}$$

   С помощью Y-комбинатора реализуйте:
   (a) Вычисление k-го простого числа.
   (b) Частичный логарифм.
   (c) Предложите три других комбинатора неподвижной точки (других — то есть, не бета-эквивалентных Y и между собой).

3. Напомним, что список может быть задан с помощью алгебраического типа с двумя конструкторами, `Nil` и `Cons` (см. доказательство неразрешимости исчисления предикатов). С учётом этого знания, и с учётом представления алгебраических типов, приведённого на лекции, реализуйте следующие конструкции:
   (a) Функцию, вычисляющую длину списка.
   (b) Функцию высшего порядка `map2` — последовательно применяет функцию к головам двух списков, возвращая список результатов: `map2 (*) [1;3] [2;4]` вернёт `[2;12]`.
   (c) Функцию `rev`, возвращающую перевёрнутый список. Например, $\mathrm{rev}[1,3,5] = [5,3,1]$.

4. Покажите, что для чёрчевских нумералов $\overline{\lambda a.\lambda b.b\ a} =_\beta \overline{a^b}$.

5. Бесконечное количество комбинаторов неподвижной точки. Дадим следующие определения

   $$\begin{aligned}
   &L := \lambda abcdefghijklmnopqstuvwxyzr.r(thisisafixedpointcombinator)\\
   &R := LLLLLLLLLLLLLLLLLLLLLLLLLL
   \end{aligned}$$

   В данном определении терм R является комбинатором неподвижной точки: каков бы ни был терм F, выполнено $R\ F =_\beta F\ (R\ F)$.
   (a) Докажите, что данный комбинатор — действительно комбинатор неподвижной точки.

(b) Пусть в качестве имён переменных разрешены русские буквы. Постройте аналогичное выражение по-русски: с 32 параметрами и осмысленной русской фразой в терме L; покажите, что оно является комбинатором неподвижной точки.

6. (задача пропущена)

7. Сформулируем определение бета-редукции на языке пред-лямбда-термов. $A \to_\beta B$, если:

   - $A \equiv (\lambda x.P)\ Q$, $B \equiv P\ [x := Q]$, при условии свободы для подстановки;
   - $A \equiv (P\ Q)$, $B \equiv (P'\ Q')$, при этом $P \to_\beta P'$ и $Q = Q'$, либо $P = P'$ и $Q \to_\beta Q'$;
   - $A \equiv (\lambda x.P)$, $B \equiv (\lambda x.P')$, и $P \to_\beta P'$.

   (a) Найдите лямбда-выражение, бета-редукция которого не может быть произведена из-за нарушения правила свободы для подстановки (для продолжения редукции потребуется производить переименование связанных переменных). Поясните, какое ожидаемое ценное свойство будет нарушено, если ограничение правила проигнорировать.

   (b) Покажите, что недостаточно наложить требования на исходное выражение, и свобода для подстановки может быть нарушена уже в процессе редукции исходно полностью корректного лямбда-выражения.

8. Как мы уже разбирали, $\not\vdash x\ x : \tau$ в силу дополнительных ограничений правила

$$\frac{}{\Gamma, x : \tau \vdash x : \tau}\ x \notin FV(\Gamma)$$

   Найдите лямбда-выражение N, что $\not\vdash N : \tau$ в силу ограничений правила

$$\frac{\Gamma, x : \sigma \vdash N : \tau}{\Gamma \vdash \lambda x.N : \sigma \to \tau}\ x \notin FV(\Gamma)$$

9. Рассмотрим подробнее отличия исчисления по Чёрчу и по Карри. Определим точно бета-редукцию в исчислении по Чёрчу: $A \to_\beta B$, если

   $A = (\lambda x^\sigma.P)\ Q$, $\quad B = P[x := Q]$
   $A = P\ Q$, $\quad B = P\ Q'$ или $B = P'\ Q$ при $P \to_\beta P'$ и $Q \to_\beta Q'$
   $A = \lambda x^\sigma.P$, $\quad B = \lambda x^\sigma.P'$ при $P \to_\beta P'$

   (a) Покажите, что в исчислении по Карри не выполняется свойство расширения типизации (subject expansion) даже в такой формулировке: если $\vdash_{\text{к}} M : \sigma$, $M \twoheadrightarrow_\beta N$ и $\vdash_{\text{к}} N : \tau$, то необязательно, что $\sigma = \tau$.

   (b) Покажите, что в исчислении по Чёрчу свойство расширения типизации в такой формулировке также не выполняется:

$$\text{найдутся такие } M, N, \sigma, \text{ что } \vdash_{\text{ч}} N : \sigma,\ M \twoheadrightarrow_\beta N, \text{ но } \not\vdash_{\text{ч}} M : \sigma.$$

   Но при этом в исчислении по Чёрчу выполнено свойство расширения типизации в такой формулировке:

$$\text{если } \vdash_{\text{к}} M : \sigma,\ M \twoheadrightarrow_\beta N \text{ и } \vdash_{\text{к}} N : \tau, \text{ то тогда } \sigma = \tau.$$

Задание №2. Просто-типизированное лямбда-исчисление

1. Докажите (по возможности строго), что предложенный на лекции алгоритм преобразования лябмда-выражения в выражение в базисе SKI действительно заканчивает работу, строя выражение, содержащее только комбинаторы SKI и скобки. В частности, ожидается, что вы полностью укажете схему индуктивного доказательства (в известном автору доказательстве две индукции с содержательными условиями — одна вложена в другую).

   Остальные части доказательства теоремы (например, что полученное выражение бета-эквивалентно исходному) доказывать не обязательно, если они очевидны, т.е. вы понимаете, что нужно доказать, какова схема доказательства и как в целом доказывать отдельные переходы.

2. Покажите, что следующая система комбинаторов образует полный базис в бестиповом лямбда-исчислении, но соответствующая им система аксиом в исчислении гильбертового типа не образует полного базиса для импликативного фрагмента:

$$\begin{aligned}
&I := \lambda x.x \\
&K := \lambda x.\lambda y.x \\
&S' := \lambda i.\lambda x.\lambda y.\lambda z.i\ (i\ ((x\ (i\ z))\ (i\ (y\ (i\ z)))))
\end{aligned}$$

Указание: покажите невыводимость $(\varphi \to \varphi \to \psi) \to (\varphi \to \psi)$.

3. Не пользуясь алгоритмом (предлагаемое алгоритмом выражение скорее всего всё равно будет громоздким), выразите следующие лямбда-термы в базисе SKI:

   (a) $F := \lambda x.\lambda y.y$ (заметим, что $T = K$);

   (b) $Not := \lambda x.x\ F\ T$;

   (c) $1 := \lambda f.\lambda x.f\ x$;

   (d) $(+1) := \lambda n.\lambda f.\lambda x.n\ f\ (f\ x)$.

4. Рассмотрим альтернативный комбинаторный базис $BCKW$, три новых комбинатора понимаются нами так:

$B := \lambda x.\lambda y.\lambda z.x\ (y\ z)$	Шейнфинкель: Z, Zusammensetzungsfunkion
$C := \lambda x.\lambda y.\lambda z.x\ z\ y$	Шейнфинкель: T, Vertauschungsfunkion
$W := \lambda x.\lambda y.x\ y\ y$	

   - Выразите I в этом базисе;
   - Выразите F в этом базисе;
   - Выразите Not в этом базисе;
   - Покажите, что данный базис позволяет выразить любой комбинатор из лямбда-исчисления;
   - Найдите схемы аксиом, соответствующие данным комбинаторам в исчислении в Гильбертовском стиле; также покажите, что импликативный фрагамент ИИВ, в котором схемы аксиом заменены на найденные вами, корректен и полон.

5. Найдите *замкнутое* выражение, в котором при редукции какого-то подвыражения вида $(\lambda.P)\ Q$ (при подстановке Q вместо переменной 0 в P) требуется изменение индексов в Q в нотации де Брауна. Предложите алгоритм для этого и покажите его корректность. Также, предложите способ, как в нотации де Брауна указывать глобальные переменные.

6. Покажите, что расширенные полиномы (см. теорему о выразительной силе просто-типизированного лямбда исчисления) выражаются в данном исчислении (т.е. для каждого E(m,n) найдётся лямбда-выражение $E : \eta \to \eta \to \eta$, что $\overline{E(m,n)} = E\ \overline{m}\ \overline{n}$). В частности, важным результатом в этом задании является лямбда-терм для умножения: $(\cdot) : \eta \to \eta$, что $(\cdot)\ \overline{m}\ \overline{n} =_\beta \overline{m \cdot n}$. Также найдите другой терм для умножения, который не имеет типа в просто-типизированном лямбда исчислении.

7. Пусть $\neg\alpha := \alpha \to \bot$. Также пусть $\varphi \leftrightarrow \psi := (\varphi \to \psi)\ \&\ (\psi \to \varphi)$. Найдите лямбда-терм в полном просто-типизированном лямбда исчислении, обитающий в типе:

   - $(\alpha \vee \beta) \to \neg(\neg\alpha\ \&\ \neg\beta)$ — правило де Моргана;
   - $(\alpha \to \beta \to \gamma) \leftrightarrow (\alpha\ \&\ \beta \to \gamma)$ — карринг;
   - $(\alpha\ \&\ (\beta \vee \gamma)) \leftrightarrow ((\alpha \vee \beta)\ \&\ (\alpha \vee \gamma))$ — дистрибутивность.

8. Обитаем ли тип $(\alpha \to \beta) \to ((\alpha \to \bot) \vee \beta)$? Ответ докажите.

Задание №3. Система F.

1. На лекции алгоритм реконструкции типов для просто типизированного лямбда-исчисления формулировался для исчисления по Карри. Сформулируйте алгоритм реконструкции типов для исчисления по Чёрчу — чем он будет отличаться от алгоритма для исчисления по Карри?

2. Постройте систему уравнений в алгебраических термах для Y-комбинатора, покажите её несовместность.

3. Задача, оставшаяся с практики 21.03.2026: в каком соотношении находятся $[\![\alpha \to \beta]\!]$ и $[\![\alpha \to \alpha]\!]$ — есть ли вложенность одного множества в другое?

4. Рассмотрим более общий тип чёрчевского нумерала для системы F: $\forall\alpha.(\alpha \to \alpha) \to (\alpha \to \alpha)$. Постройте значения этого нумерала, постройте операции сложения, умножения и возведения в степень для него, докажите их тип в системе F.

5. Выразите следующие связки через $\to, \forall$, реализуйте соответствующие функции, докажите их типы в F:

   (a) &, реализуйте функции MkPair, π_L, π_R;

   (b) $\vee$, реализуйте функции in_L, in_R, Case.

6. Рассмотрите функции. Каждая из функций использует конъюнкцию или дизъюнкцию, их можно выразить через квартор всеобщности и импликацию. Однако, получившееся выражение может не соответствовать системе HM. В связи с этим: реализуйте их в системе F и примените алгоритм W (получится ли это?); определите ранг типа этих функций; реализуйте эти функции на Хаскеле. Пользуйтесь аннотацией `{-# LANGUAGE RankNTypes #-}`, позволяющей обойти ограничения HM (за счёт потери разрешимости типовой системы языка), смотрите лекцию 5.

   (a) функция $\pi_L : \alpha \,\&\, \beta \to \alpha$ (левая проекция для упорядоченной пары) — раскройте её с учётом способа выражения конъюнкции через квантор всеобщности и импликацию.

   (b) функция $in_L : \alpha \to \alpha \vee \beta$

   (c) `let` $id = \lambda x.x$ `in` $\langle$`id 0, id “a”`$\rangle$

7. Модифицируйте алгоритм W, чтобы включить в него работу с упорядоченными парами и алгебраическими типами.

8. Запишите тип списка (из целых чисел, используйте константу `int`) в HM, а также реализуйте функции для получения его длины и функцию map. Используйте расширения (типизацию для Y-комбинатора, мю-оператор в варианте изорекурсивных типов).

9. Выразите Y комбинатор в Хаскеле (Окамле), укажите его тип.

10. Рассмотрим QuickSort:

```
let rec quick l =  match l with
    [] -> []
  | l1 :: ls -> List.filter (fun x -> x < l1) ls @ [l1] @ List.filter (fun x -> x >= l1)
```

    Укажите полные типовые схемы в системе HM для всех функций, участвующих в данном примере (тип списка раскрывать не надо).

11. Заметим, что список — это «параметризованные» числа в аксиоматике Пеано. Число — это длина списка, а к каждому штриху мы присоединяем какое-то значение. Операции добавления и удаления элемента из списка — это операции прибавления и вычитания единицы к числу.

    Рассмотрим тип «бинарного списка»:

```
type 'a bin_list = Nil | Zero of (('a*'a) bin_list) | One of 'a * (('a*'a) bin_list);;
```

    Заметим, что здесь мы рассматриваем двоичную запись числа (чередующиеся `Zero` и `One`) — двоичную запись длины бинарного списка, и элемент двоичной записи номер n хранит 2^n или $2^n + 1$ значение (в зависимости от типа элемента). Например, 5-элементный список:

```
One ("a", Zero (One ((("b","c"),("d","e")), Nil)))
```

    По идее, операция добавления элемента к списку записывается на языке Окамль вот так (сравните с прибавлением 1 к числу в двоичной системе счисления):

```
let rec add elem lst = match lst with
    Nil -> One (elem,Nil)
  | Zero tl -> One (elem,tl)
  | One (hd,tl) -> Zero (add (elem,hd) tl)
```

Однако, тип этой функции Окамль вывести автоматически не может, его надо указывать явно, чтобы код скомпилировался:

```
let rec add : 'a . 'a -> 'a bin_list -> 'a bin_list = fun elem lst -> match lst with
```

(a) Какой тип имеет `add` в (расширенной) системе F (напомним, поскольку функция рекурсивна, она должна использовать Y-комбинатор в своём определении)? Считайте, что семейство типов `bin_list 'a` предопределено и обозначается как τ_α. Также считайте, что определены функции roll и unroll с надлежащими типами. Какой ранг имеет тип этой функции? Почему этот тип не выразим в типовой системе Хиндли-Милнера?

(b) Предложите функции для печати списка и для удаления элемента списка (головы).

(c) Предложите функцию для эффективного соединения двух списков (источник для вдохновения — сложение двух чисел в столбик).

(d) Предложите функцию для эффективного выделения n-го элемента из списка.

12. Предложите выражение на языке C++ (возможно, использующее шаблоны), имеющее следующий род (тип):

(a) $\star \to \star \to \star$; $\star \to \mathbf{unsigned}$

(b) $\mathbf{int} \to (\star \to \star)$

(c) $(\star \to \mathbf{int}) \to \star$

(d) $\Pi x^\star.\lambda n^{\mathbf{int}}.F(n,x)$, где

$$F(n,x) = \begin{cases} \mathbf{int}, & n = 0 \\ x \to F(n-1,x), & n > 0 \end{cases}$$

Задание №4. Язык Аренд.

1. Докажите, приведя компилирующуюся программу на языке Аренд (возможно, вам потребуются функции и приёмы, изложенные в документации по языку: `https://arend-lang.github.io/documentation/tutorial/PartI/`):

(a) коммутативность умножения;

(b) дистрибутивность: $(a+b)\cdot c = a\cdot c + b\cdot c$;

(c) куб суммы: $(a+1)^3 = a^3 + 3\cdot a^2 + 3\cdot a + 1$.

2. Определим, что x делится на p, если обитаем тип `\Sigma (q : Nat) (p * q = x)`.

(a) Покажите, что если x делится на 6, то x делится и на 3;

(b) Покажите, что $x!$ делится на x;

(c) Покажите, что если x делится на y и y делится на z, то x делится на z;

3. Определите предикат (т.е. функцию с надлежащим типом) для формализации понятия простого числа `isPrime`. Покажите, что:

(a) 3 и 11 — простые числа;

(b) Произведение простых чисел непросто;

(c) 2 — единственное чётное простое число.

4. Определим отношение «меньше» на натуральных числах так (с помощью индуктивного типа, обобщения алгебраического типа данных — зависимого типа, в котором при разных значениях аргументов типа допустимы разные конструкторы):

```
\data NatLessEq (a b : Nat) \with
  | 0, m => natlesseq-zero
  | suc m, suc n => natlesseq-next (NatLessEq m n)
```

Например, конструктор `natlesseq-zero` можно использовать только если первый аргумент типа — число 0. А конструктор `natlesseq-next` применим только если первый аргумент больше 0; при этом данный конструктор требует значение типа с определёнными аргументами в качестве своего аргумента.

Будем говорить, что $a \preceq b$ тогда и только тогда, когда `NatLessEq a b` обитаем. Например, утверждение $1 \preceq 3$ доказывается так:

```
\func one-le-three : NatLessEq 1 3 => natlesseq-next (natlesseq-zero)
```

В самом деле, `natlesseq-zero` может являться конструктором типа `NatLessEq 0 b`, а тогда

```
natlesseq-next (natlesseq-zero) : NatLessEq 1 (b+1)
```

Унифицировать $b+1$ и 3 компилятор (в данном случае) может самостоятельно, и потому код выше проходит проверку на корректность.

Докажите (везде предполагается, что a, b, c : `Nat`, если не указано иного):

(a) $a \preceq b$ тогда и только тогда, когда a меньше или равно b в смысле натуральных чисел (здесь требуется рассуждение на мета-языке).

(b) $a \preceq a + b + 1$; то есть, определите функцию
`\func n-less-sum (a b : Nat) : NatLessEq a (a Nat.+ suc b)`

(c) Если $a \preceq b$, то $a + c \preceq b + c$

(d) Если $a \preceq b$ и $c \preceq d$, то $a \cdot c \preceq b \cdot d$

(e) $a \preceq 2^a$

(f) Транзитивность: если $a \preceq b$ и $b \preceq c$, то $a \preceq c$

(g) $a \preceq b \vee b \preceq a$

(h) Найдите стандартное определение отношения «меньше» в библиотеке Аренда (`Nat.<`) и докажите, что $a \preceq b$ тогда и только тогда, когда $a < b$ или $a = b$ (реализуйте функции
`there (p : NatLessEq a b) : Data.Or (a Nat.< b) (a = b)` и обратную к ней).

(i) Покажите, что $a \preceq b$ тогда и только тогда, когда $\exists k^{\mathbb{N}_0}.a + k = b$.

5. С точки зрения изоморфизма Карри-Ховарда индуктивные типы можно воспринимать как аналоги предикатов. В этом задании надо построить индуктивные типы для различных предикатов:

(a) Факториал (`IsFact n`), который будет обитаем только для таких n, что $n = k!$. Докажите на языке Аренд, что `IsFact` $(1 \cdot 2 \cdot 3 \cdot \dots \cdot n)$ всегда обитаем, а тип `IsFact 3` — необитаем.

(b) Наибольший общий делитель двух чисел `GCD x a b`; *указание/пожелание:* воспользуйтесь алгоритмом Эвклида.

(c) Ограниченное натуральное число `IndFin n`, обитателями типа являются только те числа, которые меньше n. В стандартной библиотеке `Fin` определён через натуральные числа; сделайте это исключительно через индуктивные типы. Покажите, что если `x : IndFin m` и `y : IndFin n`, то `x + y : IndFin (m + n)`.

6. Рассмотрим следующее доказательство уникальности элементов списка:

```
\func not-member (A : \Type) (elem : A) (l : List A) : \Type \elim l
  | nil => \Sigma
  | :: hd tl => \Sigma (Not (hd = elem)) (not-member A elem tl)

\func unique-list (A : \Type) (l : List A) : \Type \elim l
  | nil => \Sigma
  | :: hd tl => \Sigma (not-member A hd tl) (unique-list A tl)
```

Докажем, что список $[0, 1, 2]$ состоит из уникальных элементов:

```
\func r-unique => unique-list Nat (0 :: 1 :: 2 :: nil)
\func x : r-unique => ((contradiction, (contradiction, ())), ((contradiction, ()), (() , ())))
```

(a) Напишите функцию, строящую список `b` натуральных чисел от 0 до n и доказательство `unique-list Nat b`.

(b) Покажите, что если $a_0 < a_1 < \dots < a_n$, то список $[a_0, a_1, \dots, a_n]$ уникальный.

(c) Покажите, что если $n \geq 2$ и $a_k \neq a_{k+1}$ при $0 \leq k < n$, то необязательно, что список $[a_0, a_1, \dots, a_n]$ — уникальный.

7. Рассмотрим правый дальний нижний угол λ-куба ($\{(\star,\star);(\star,\Box);(\Box,\star)\}$). Можно предположить, что тогда в такой системе возможны и функции рода $f : \star \to \star$ (как композиция функций $p : \star \to \alpha$ и $q : \alpha \to \star$ — например, можно кодировать тип его именем, затем по имени типа восстанавливать сам тип обратно). Почему всё-таки такие функции в обобщённых типовых системах невозможны без четвёртого элемента $(\Box,\Box)$?

8. Какова должна быть топология на множестве пар натуральных чисел (интуитивно мы будем понимать эти пары как рациональные числа, пары «числитель-знаменатель»), чтобы непрерывными были бы те и только те функции, для которых выполнено $f(p,q) = f(p',q')$ для всех таких p, p', q и q', что $p \cdot q' = p' \cdot q$. Напомним, что равенство мы понимаем как наличие непрерывного пути между точками.

Задание №5. Дополнительные задачи на доказательства. Равенство.

1. Рассмотрим модуль числа, $|x|$.
   (a) Предложите определение этой конструкции (должна отображать `Int` в `Nat`). Докажите, что $||x|| = |x|$.
   (b) Докажите, что если $x \geq 0$, то $|x| = x$.
   (c) Докажите, что $|a-b| + |a+b| = 2 \cdot \max(|a|,|b|)$.

2. На практике было много вопросов по поводу доказательства $\forall x \in \mathbb{N}.\forall y \in \mathbb{N}.x < y \to \neg\exists z \in \mathbb{N}.x = z \cdot y$. Рассмотрим несколько упражнений (дополнительных лемм), которые могут помочь в этом доказательстве.
   (a) Заметим, что это утверждение неверно при $x, y, z \in \mathbb{N}_0$. Поэтому, предложите контрпример и докажите его на языке Аренд.
   (b) Рассмотрим деление с остатком, divmod:
   `\divmod (a b : Nat) (b /= 0) => \Sigma (q r : Nat) (a = q * b + r) (r < b)`. Будем обозначать первый элемент пары (собственно деление) как // (в стиле языка Питон). Докажите, что $a_1//b \geq a_2//b$, если $a_1 > a_2$.
   (c) Докажите, что $a_1//b > a_2 \div b$, если $a_1 \geq a_2 + b$.
   (d) Докажите, что $a//b_1 \geq a \div b_2$, если $b_1 < b_2$.
   (e) Докажите, что если $a_1//b = a_2//b$, то $|a_2 - a_1| < b$.
   (f) Докажите, что если $(p1, q1, r1) =$ `divmod` a b и $(p2, q2, r2) =$ `divmod` a b, то $p1 = p2$ и $q1 = q2$.
   (g) Докажите, что если $a//b = 1$, то $a < 2b$ (ну, или $a = 1 \cdot b + k, k < b$).
   (h) Докажите, что если $a//b = 0$, то $a < b$ и a `mod` $b = a$.
   (i) Докажите, что если $a < b$, то a `mod` $b = a$ и $a//b = 0$.

3. Как вы помните из лекции, в языке Аренд существует иерархия вложенных типовых универсумов. Если в типе отсутствует упоминание `\Type`, то данный тип принадлежит универсуму 0. Однако, если в типе упоминается `\Type k`, то тип принадлежит универсумам, не меньшим $k+1$. Уровень универума обозначается специальным ключевым словом `\lp`. Над индексами можно проводить простые операции и сопоставление с образцом (`\suc\lp`). Более подробно можно это прочесть в документации по языку Аренд:

   `https://arend-lang.github.io/documentation/tutorial/PartI/universes.html`

   Рассмотрим определения:

```
\func ChurchT (x : \Type) => (x -> x) -> (x -> x)
\func Church => \Pi (x : \Type) -> ChurchT x
\func Zero : Church => \lam t f x => x

\func incT (t : \Type) (n : ChurchT t) => \lam f x => n f (f x)
\func pair_plus (type : \Type) (pair : \Sigma (ChurchT type) (ChurchT type)) :
  \Sigma (ChurchT type) (ChurchT type) => (pair.2, incT type pair.2)
\func dec (n : Church) : Church => \lam (t : \Type) =>
    (n (\Sigma (ChurchT t) (ChurchT t)) (pair_plus t) (Zero t, Zero t)).1
\func sub (a : Church (\suc\lp)) (b : Church) => a (Church \lp) dec b
```

Определите, развивая определения выше:

(a) Операцию умножения.

(b) Операцию «деление на три» (естественно, в версии, не использующей Y-комбинатор).

(c) Операцию возведения в степень, определявшуюся как $\lambda m.\lambda n.n\ m$.

(d) Деление.

(e) Вычисление факториала.

4. Введём тип данных `IsEven`:

```
\data IsEven (n : Nat) \elim n
  | 0 => zero-is-even
  | (suc (suc k)) => next-next (IsEven k)
```

Доказать, что этот тип является утверждением, можно например так:

```
\func all-even-different (n : Nat) (a b : IsEven n) : a = b \elim n, a, b
  | 0, zero-is-even, zero-is-even => idp
  | suc (suc n), next-next a, next-next b => pmap next-next (all-even-different n a b)

\func is-even-isProp (n : Nat) : isProp (IsEven n) => all-even-different n
```

Обратите внимание: по переменным n, a и b производится *элиминация*, то есть множество значений переменных разбивается на фрагменты (в соответствии с конструкторами типа данных), и доказательство утверждения проводится независимо для каждого фрагмента; в частности, цель доказательства изменяется и просходит замена переменных a и b на сопоставляемые варианты (вместо ожидаемого типа `a = b` мы ожидаем тип `zero-is-even = zero-is-even`, и т.п.). Сравните с лекцией про элиминаторы, каждый из вариантов — тело отдельной функции-элиминатора.

Чтобы воспроизвести тот же эффект в конструкции `\case`, нужно указывать ключевое слово `\elim` перед каждой элиминируемой переменной:

```
\case \elim n, \elim a, \elim b \with { ... }
```

Полный код, определяющий тип `IsEven` (вместе с доказательством того, что тип — утверждение), выглядит так:

```
\data IsEven (n : Nat) \elim n
  | 0 => zero-is-even
  | (suc (suc k)) => next-next (IsEven k)
  \where {
    \func all-even-different ... -- скопируйте код функции сюда
    \use \level is-even-isProp (n : Nat) : isProp (IsEven n) => all-even-different n
  }
```

Однако, незавершённым остаётся доказательство разрешимости типа `IsEven n`. Восполните лакуны:

```
\func even-is-dec (a : Nat) : Dec (IsEven a) \elim a
  | 0 => yes zero-is-even
  | 1 => no {?}
  | suc (suc a) => {?}
```

5. Справедливости ради, в предыдущем задании компилятор сам может догадаться, что `IsEven` — утверждение. Но далеко не для всех типов это очевидно, и тогда функция с префиксом `\use \level` становится необходимой. Например, в следующем типе «простое число» данная функция должна доказать, что любые два значения типа при данном x равны:

```
\data Div3 (x : Nat)
  | remainder-zero (Exists (p : Nat) (p Nat.* 3 = x))
  | remainder-one (Exists (p : Nat) (p Nat.* 3 Nat.+ 1 = x))
  | remainder-two (Exists (p : Nat) (p Nat.* 3 Nat.+ 2 = x))
  \where \use \level levelProp {x : Nat} (a b : Div3 x) : a = b => {?}
```

(a) Замените `{?}` в тексте выше на корректное доказательство.

(b) Определите функцию, которая бы по x и значению $\exists pq.3 \cdot p + q = x \,\&\, 0 \leq q < 3$ возвращала бы `Div3 x` (понятно, можно разделить x на 3, но нам уже результат деления дали вторым аргументом — задача в том, чтобы им воспользоваться).

(c) Постройте аналогичный тип `Prime x` для простых чисел — с тремя вариантами `less-than-two`, `is-prime`, `is-composite` — и определите функцию, строящую по числу значение данного типа.

(d) Покажите, что в типе `Prime (x*x + 2*x + 1)` всегда (кроме граничных случаев) обитает вариант `is-composite`.

(e) Покажите, что в типе

```
\data SuperDec (P : \Prop)
| sure P
| nope (P -> Empty)
| neither ((P || (P -> Empty)) -> Empty)
\where \use \level superDecProp {P : \Prop} (a b : SuperDec P): a = b => {?}
```

вариант `neither` невозможен (также, заполните пропущенное доказательство `superDecProp`).